E-COMMERCE PROJECT REVISION NOTES

Project Overview

This is a backend e-commerce application built with Spring Boot, Spring Security, JWT, Spring Data JPA, MySQL, Lombok, ModelMapper, and Maven.

Main features:
- User signup/login with JWT cookie authentication
- Role-based access: public, authenticated user, admin
- Category management
- Product management with image upload
- Cart management
- Address management
- Order placement
- Payment information storing
- Global exception handling
- Pagination and sorting for category/product APIs

Root Directory

pom.xml
Maven configuration file. It defines dependencies like Spring Web, JPA, Security, Validation, MySQL, JWT, Lombok, ModelMapper, and testing.

mvnw, mvnw.cmd
Maven wrapper files. They allow running Maven commands without installing Maven globally.

images/
Stores uploaded product images.

target/
Build output folder. Contains compiled class files and generated JAR. Not written manually.

src/main/java/
Main Java source code.

src/main/resources/
Application configuration files.

src/test/
Test code and test configuration.

Main Package: org.example.ecommerce

EcommerceApplication.java
Main Spring Boot startup class. It contains the main() method and starts the application.

config Directory
Used for application-level configuration.

AppConfig.java
Creates common beans:
- ModelMapper for converting Entity to DTO and DTO to Entity.
- CommandLineRunner to seed default roles into the database from AppRole.

AppConstants.java
Stores reusable constants like default page number, page size, sorting fields, and sorting direction.

SecurityConfig.java
Main Spring Security configuration.
- Disables CSRF.
- Makes the app stateless.
- Allows public access to /api/auth/**, /api/public/**, /images/**.
- Restricts /api/admin/** to users with ADMIN role.
- Adds JWT filter before Spring's username/password filter.
- Provides PasswordEncoder, AuthenticationManager, and authentication provider.

controller Directory
Controllers expose REST API endpoints. They receive HTTP requests and call service classes.

AuthController.java
Handles authentication:
- /api/auth/signin for login
- /api/auth/signup for registration
- /api/auth/signout for logout
- /api/auth/user for current user details
- Uses JWT cookie authentication.

CategoryController.java
Handles category APIs:
- Public category listing
- Admin category create, update, delete

ProductController.java
Handles product APIs:
- Add product under category
- Get all products
- Get products by category
- Search products by keyword
- Update/delete product
- Upload/update product image

CartController.java
Handles cart APIs:
- Add product to cart
- View logged-in user cart
- Admin view all carts
- Update product quantity
- Remove product from cart

AddressController.java
Handles address APIs:
- Create address
- Get all addresses
- Get address by ID
- Get logged-in user’s addresses
- Update/delete address

OrderController.java
Handles order placement using cart, address, and payment details.

model Directory
Models are JPA entity classes. These classes map to database tables.

User.java
Represents application user. Fields include username, email, password, roles, addresses, cart, and products. Connected with many-to-many Role, one-to-many Address, one-to-one Cart, and one-to-many Product.

Role.java
Represents user role in database, such as admin or user.

AppRole.java
Enum for role names.

Category.java
Represents product category. One category has many products.

Product.java
Represents product details like name, description, quantity, image, price, discount, special price. Connected to Category, seller User, and CartItem.

Cart.java
Represents user cart. One user has one cart. Contains cart items and total price.

CartItem.java
Represents one product inside a cart with quantity, discount, and product price.

Address.java
Stores user address details like street, building name, city, state, country, and pincode.

Order.java
Represents placed order. Contains email, order items, order date, payment, total amount, status, and address.

OrderItem.java
Represents individual product inside an order.

Payment.java
Stores payment information like payment method, payment gateway name, payment ID, status, and response message.

payload Directory
Payload classes are DTOs. They are used for request and response data instead of exposing entity classes directly.

Important DTOs:
- ProductDTO
- CategoryDTO
- CartDTO
- CartItemDTO
- AddressDTO
- OrderDTO
- OrderItemDTO
- OrderRequestDTO
- PaymentDTO
- APIResponse

Response wrapper classes:
- ProductResponse contains product list plus pagination data.
- CategoryResponse contains category list plus pagination data.

Interview line:
DTOs help separate internal database entities from API request/response structure.

repositories Directory
Repositories interact with the database using Spring Data JPA.

All repositories extend JpaRepository, so they automatically get methods like save(), findById(), findAll(), and delete().

Important repositories:
- UserRepository: finds user by username, checks username/email existence.
- RoleRepository: finds role by role name.

- CategoryRepository: finds category by category name.
- ProductRepository: finds products by category and keyword with pagination.
- CartRepository: finds cart by user email and cart ID.
- CartItemRepository: finds cart item by product and cart.
- OrderRepository, OrderItemRepository, PaymentRepository, AddressRepository.

service Directory
Service layer contains business logic. Controllers call services, services call repositories.

CategoryService / CategoryServiceImpl
Handles category CRUD, pagination, sorting, duplicate checks.

ProductService / ProductServiceImpl
Handles product creation, update, delete, search, category filter, pagination, image update.

CartService / CartServiceImpl
Handles add to cart, update quantity, remove product, calculate total price.

AddressService / AddressServiceImpl
Handles address creation, update, delete, and fetching user addresses.

OrderService / OrderServiceImpl
Places order from user cart, creates order items, payment entry, updates order status.

FileService / FileServiceImpl
Handles product image upload into the images/ folder.

Interview line:
The service layer keeps business logic separate from controllers, making the code cleaner and easier to test.

security Directory
Contains authentication-related request/response classes and JWT logic.

SignupRequest.java
DTO for user registration.

MessageResponse.java
Simple response message class.

jwt/LoginRequest.java
DTO for login request.

jwt/UserInfoResponse.java
Response after login/current user API. Contains user info and roles.

jwt/JwtUtils.java
Creates, validates, reads, and clears JWT cookies.

jwt/AuthTokenFilter.java
Runs on every request. It reads JWT from cookie, validates it, loads user details, and sets authentication in Spring Security context.

jwt/AuthEntryPointJwt.java
Handles unauthorized access errors.

security/services/UserDetailsImpl.java
Custom implementation of Spring Security UserDetails.

security/services/UserDetailsServiceImpl.java
Loads user from database by username for authentication.

exceptions Directory
Handles custom and global exceptions.

ResourceNotFoundException.java
Thrown when an entity like product/category/address is not found.

APIException.java
Custom business exception.

MyGlobalExceptionHandler.java
Centralized exception handler using @RestControllerAdvice. Handles validation errors, resource not found errors, and API exceptions.

Interview line:
Global exception handling avoids repeated try-catch blocks in controllers and gives consistent API error responses.

util Directory
AuthUtil.java
Utility class to get currently logged-in user, user ID, or email from Spring Security context.

resources Directory
application.properties
Main application configuration:
- App name
- MySQL datasource
- JWT secret, expiration, cookie name
- Image upload path

- Logging levels
- Server port 5000

application-mysql.properties
Profile-specific MySQL configuration for local database.

Important interview note:
Do not say actual passwords or secrets. Say: Database credentials and JWT secrets are configured through properties/environment variables.

Typical Request Flow: Add Product To Cart
1. User logs in through AuthController.
2. Server creates JWT cookie.
3. User calls cart API.
4. AuthTokenFilter validates JWT.
5. Controller receives request.
6. Controller calls CartService.
7. Service gets product/user/cart from repositories.
8. Service updates cart item and total price.
9. Repository saves data in MySQL.
10. Controller returns CartDTO.

Architecture Summary
Controller layer: handles HTTP requests
Service layer: business logic
Repository layer: database operations
Model layer: JPA entities
Payload layer: DTOs
Security layer: authentication and authorization
Exception layer: centralized error handling
Config layer: beans, constants, security setup

Best Interview Explanation
My project is a Spring Boot based e-commerce REST API. I used Spring Web for APIs, Spring Data JPA for database operations, MySQL as the database, Spring Security with JWT cookies for authentication, and role-based authorization for admin routes. The project follows layered architecture where controllers handle requests, services contain business logic, repositories interact with the database, entities map to tables, and DTOs are used for clean API responses.